Solution: Check Prime

Problem Statement

Write a Recursive Solution to Check if a Given Number is a Prime Number or Not.

Given a positive integer, we need to determine whether it is a prime number or not. A prime number is a number greater than 1 that has no positive divisors other than 1 and the number itself.

The following table describes three example outputs for three inputs along with the description:

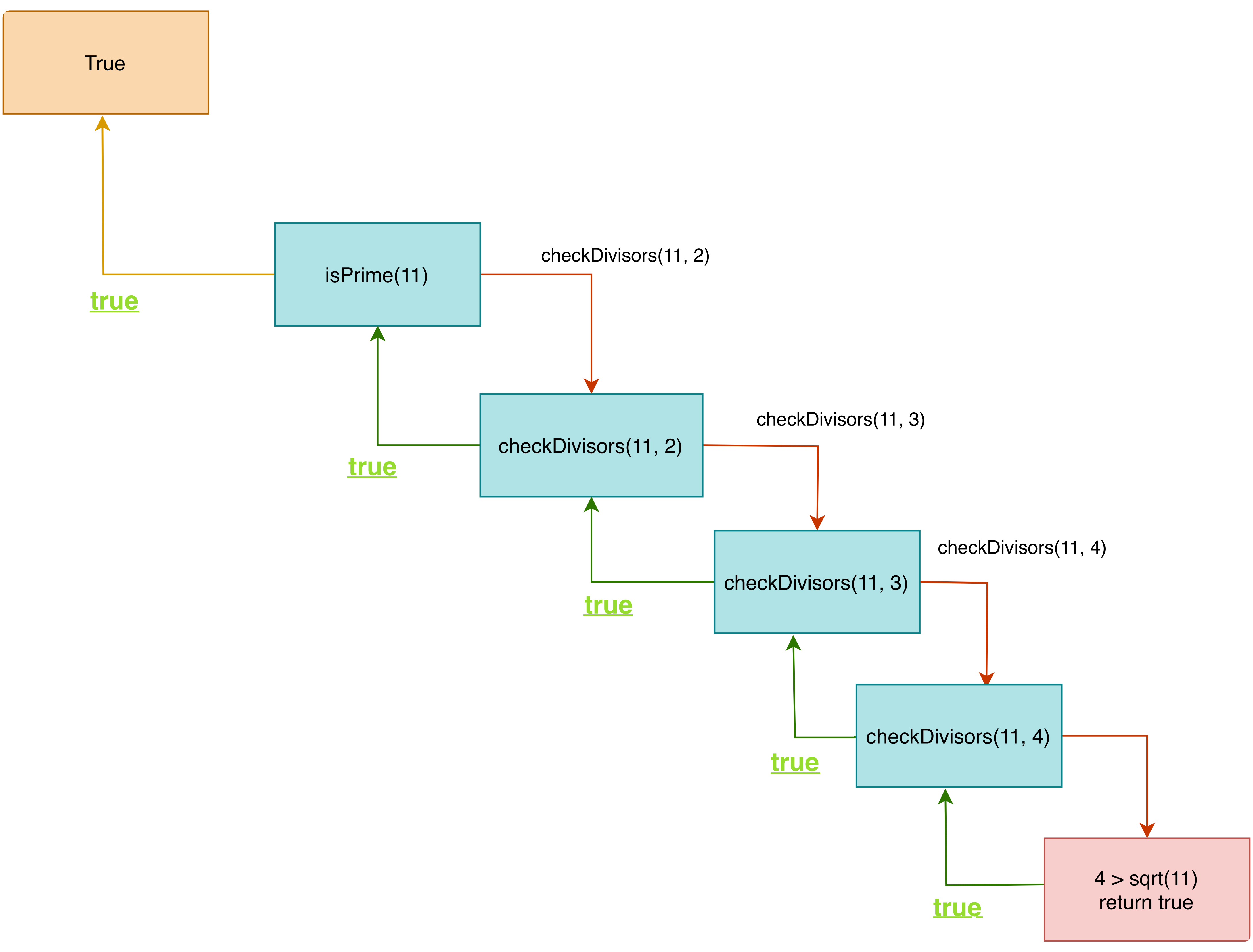
Input(s)	Output(s)	Explanation
Number = 7	Is Prime = true	The number 7 is only divisible by 1 and 7, so it is a prime number.
Number = 12	Is Prime = false	The number 12 is divisible by 1, 2, 3, 4, 6, and 12, so it is not a prime number.
Number = 23	Is Prime = true	The number 23 is only divisible by 1 and 23, so it is a prime number.

Solution

The algorithm to check if a number is prime or not using recursion can be defined as follows:

1. If the number is less than 2, return false (base case).
2. If the number is equal to 2, return true (base case).
3. If the number is divisible by any number from 2 to the square root of the number, return false.
4. Otherwise, return true.

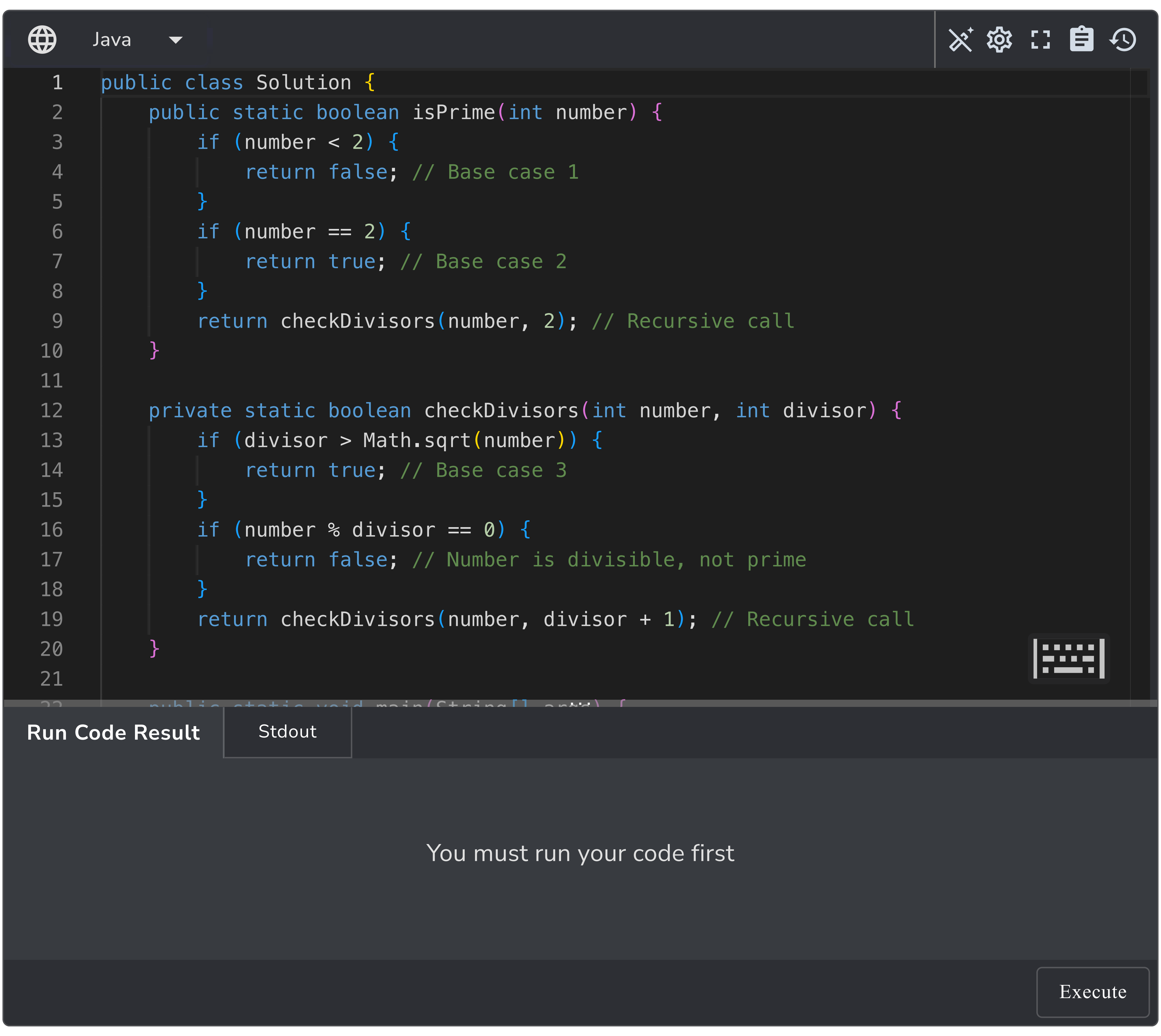
The recursive approach works by checking if the number has any divisors from 2 to its square root. If it does, the number is not prime. If no divisors are found, the number is prime.



Example Dry run to check if 11 is a prime number or not

Code

Here is the code for this algorithm:



```java
public class Solution {
    public static boolean isPrime(int number) {
        if (number < 2) {
            return false; // Base case 1
        }
        if (number == 2) {
            return true; // Base case 2
        }
        return checkDivisors(number, 2); // Recursive call
    }

    private static boolean checkDivisors(int number, int divisor) {
        if (divisor > Math.sqrt(number)) {
            return true; // Base case 3
        }
        if (number % divisor == 0) {
            return false; // Number is divisible, not prime
        }
        return checkDivisors(number, divisor + 1); // Recursive call
    }

```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(sqrt(N))$, where N is the given number. This is because the algorithm checks the divisibility of the number up to its square root. The space complexity is $O(sqrt(N))$ as well, as the maximum depth of the recursion is determined by the square root of the number.

← Previous
5. Check Prime

Next →
6. Perfect Square

☐ Mark as Completed